

ОБЕСПЕЧЕНИЕ КАЧЕСТВА ФУНКЦИОНИРОВАНИЯ КОМПЬЮТЕРНЫХ СИСТЕМ

Практическая работа: покрытие, дефект и приоритизация

Результат: вы исследуете неполное покрытие, создаёте воспроизводимый отчёт о дефекте и обосновываете его приоритет.

ОРИЕНТИРЫ

На чём сосредоточиться

- Факт вместо общего впечатления.
- Проверяемый ожидаемый результат.
- Связь проверки с риском.

ПОНЯТИЯ

Рабочий словарь

Severity

Тяжесть последствия для системы или пользователя.

Priority

Очередность исправления с учётом контекста поставки.

Доказательство

Вывод теста, журнал или скриншот, подтверждающий наблюдение.

Ветка

Один из возможных путей выполнения условия.

КОНТЕКСТ

Учебный кейс

Тесты не проверяют экспресс-доставку. При этом пользователи получают бесплатную доставку для маленьких заказов. Зафиксируйте проблему так, чтобы разработчик мог воспроизвести и исправить её.

ПРИМЕР

Данные для проверки

```
def delivery_price(total: int, method: str) -> int:
    if total < 0:
        raise ValueError("Сумма не может быть отрицательной")
    if total >= 3000:
        return 0
    if method == "express":
        return 500
    return 250
```

В исходном наборе есть тесты только для обычной доставки.

РАЗБОР

Главная мысль

Работайте по инструкции, фиксируя входные данные, действие и наблюдаемый результат.

Сравнивайте готовый артефакт с критериями самопроверки.

ДЕЙСТВИЕ

Практический маршрут

- Составьте список ветвей функции и отметьте непроверенные.
- Напишите тест, который воспроизводит неверную цену экспресс-доставки.
- Оформите отчёт о дефекте с точными входными данными.
- Оцените тяжесть и приоритет отдельно; объясните разницу.
- Добавьте регрессионную проверку после предложенного исправления.

САМОПРОВЕРКА

Перед тем как сделать вывод

- Указаны ли конкретные входные данные?
- Можно ли повторить проверку?
- Связано ли решение с риском качества?